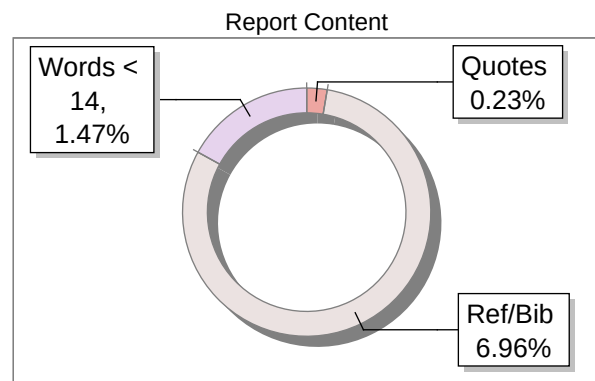
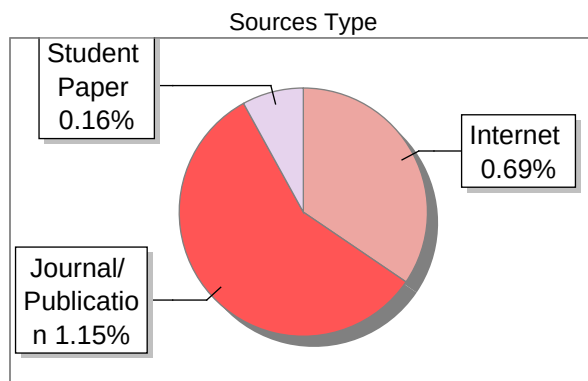
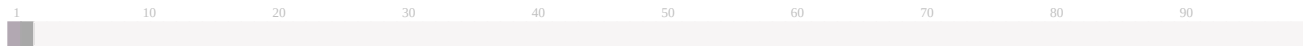


### Submission Information

|                          |                                                       |
|--------------------------|-------------------------------------------------------|
| Author Name              | Darshan                                               |
| Title                    | Intelligent Agile Project Management System Using NLP |
| Paper/Submission ID      | 5670948                                               |
| Submitted by             | lib@pesce.ac.in                                       |
| Submission Date          | 2026-05-18 12:39:33                                   |
| Total Pages, Total Words | 36, 5644                                              |
| Document type            | Research Paper                                        |

### Result Information

Similarity **2 %**

### Exclude Information

|                             |              |
|-----------------------------|--------------|
| Quotes                      | Not Excluded |
| References/Bibliography     | Excluded     |
| Source: Excluded < 14 Words | Not Excluded |
| Excluded Source             | <b>0 %</b>   |
| Excluded Phrases            | Not Excluded |

### Database Selection

|                        |         |
|------------------------|---------|
| Language               | English |
| Student Papers         | Yes     |
| Journals & publishers  | Yes     |
| Internet or Web        | Yes     |
| Institution Repository | Yes     |

A Unique QR Code use to View/Download/Share Pdf File





## DrillBit Similarity Report

2

SIMILARITY %

10

MATCHED SOURCES

A

GRADE

A-Satisfactory (0-10%)

B-Upgrade (11-40%)

C-Poor (41-60%)

D-Unacceptable (61-100%)

| LOCATION | MATCHED DOMAIN                                                                              | %  | SOURCE TYPE   |
|----------|---------------------------------------------------------------------------------------------|----|---------------|
| 1        | resources.collab.net                                                                        | <1 | Internet Data |
| 2        | Challenges in Enterprise Adoption of Agile Methods - A Survey by Mahanti-2006               | <1 | Publication   |
| 3        | ieomsociety.org                                                                             | <1 | Publication   |
| 4        | Thesis Submitted to Shodhganga Repository                                                   | <1 | Publication   |
| 5        | 1library.co                                                                                 | <1 | Internet Data |
| 6        | From Inspiring Change to Directing Change How Ethnographic Praxis Can Move beyo by HOU-2015 | <1 | Publication   |
| 7        | rsisinternational.org                                                                       | <1 | Publication   |
| 8        | Submitted to Visvesvaraya Technological University, Belagavi                                | <1 | Student Paper |
| 9        | Twitter Versus the Traditional Media by Morris-2017                                         | <1 | Publication   |
| 10       | www.tability.io                                                                             | <1 | Internet Data |

# CHAPTER 1

## INTRODUCTION

Software delivery today is overwhelmingly Agile. Teams plan in sprints, track work on Kanban boards, and rely on rapid feedback to ship features incrementally. Yet the very speed that makes Agile attractive also creates noise: hundreds of tasks, shifting priorities, and a steady stream of bug reports from testers, users, and developers. Two problems show up again and again in student projects, startups, and mid-sized product teams alike.

First, duplicate bugs waste time. The same crash or UI glitch gets reported three or four times because each person phrases it differently. QA spends hours matching issues manually; developers fix one ticket while three others stay open. Second, schedule risk is discovered too late. A task crosses its deadline quietly while still sitting "In Progress" on the board. By the time the sprint review arrives, the spillover is already baked in.

Traditional project management <sup>9</sup> platforms were not designed with lightweight, AI-first triage in mind. Tools like Jira, ClickUp, and Azure DevOps excel at workflow configuration and enterprise governance, but they can feel heavy for teams that only need solid sprint tracking, bug management, and a clear view of what is slipping [13] - [15]. Keyword search for duplicates fails when descriptions use different vocabulary. Static dashboards show status but rarely warn before a deadline is breached in a way that is immediate and understandable to every role on the team.

Recent IEEE literature confirms both the opportunity and the gap. Agile adoption continues to grow across industries, yet scaling practices in larger organizations remains challenging [1], [4]. Studies <sup>2</sup> show that Agile practices can improve quality and productivity when applied consistently [3], but teams still need better decision support at the point of work not in a separate analytics portal. Machine learning for sprint performance, issue resolution time, and effort prediction has matured [8] - [12], and systematic reviews highlight AI as a promising direction for project management [7]. What is still missing, especially for resource-constrained teams, is a single affordable platform that combines semantic duplicate detection, Agile boards, role-based notifications, and interpretable delay visibility in one deployable product.

This project, the AI-Enhanced Agile Project Management System, was built to fill that gap. It is a full-stack application our team designed, trained, and integrated end to end: testers report bugs and see duplicate warnings before they submit; managers merge confirmed duplicates; developers drag tasks across a five-column Kanban; and everyone views delay analytics on a central dashboard. The intelligence is not bolted on as an afterthought it runs inside the same FastAPI backend that powers authentication, projects, sprints, tasks, and notifications.

The following sections state our aim, numbered objectives, analysis of existing approaches, and the proposed solution in the same structured manner used in our reference project documentation for final-year evaluation.

## 1.1 AIM

The aim of this project is to design and implement an AI-enhanced Agile project management platform that helps software teams work smarter not harder by:

- (a) Automatically detecting likely duplicate bug reports using a fine-tuned Sentence-BERT embedding model and cosine similarity, so triage happens at submission time rather than days later; and
- (b) Surfacing schedule risk through transparent deadline-based delay detection, priority escalation, dashboard analytics, and notifications so delayed work is visible while there is still time to react.

The system must remain approachable for small and medium-sized teams: a clean React interface, a single FastAPI service, MongoDB storage, and open-source components that can run on a laptop or a modest cloud VM without Kubernetes or paid SaaS subscriptions.

## **1.2 OBJECTIVES**

The following objectives guided development and evaluation:

### **1. User Authentication and Role-Based Access**

Implement secure registration and login with JWT tokens and bcrypt password hashing. Enforce four roles Admin, Manager, Developer, and Tester with permissions aligned to real Agile responsibilities (project creation, sprint planning, task execution, bug reporting).

### **2. Agile Project, Sprint, and Task Management**

Support project CRUD, member management, sprint lifecycle, backlog assignment, and a five-column Kanban board (Backlog → Todo → In Progress → Review → Done) with drag-and-drop status updates.

### **3. Intelligent Bug Deduplication at Submission Time**

Fine-tune Sentence-BERT (all-MiniLM-L6-v2) on balanced duplicate/non-duplicate pairs; deploy inference via POST /ai/check-duplicate; store 384-dimensional embeddings in MongoDB; warn users in the UI when similarity exceeds threshold 0.45.

### **4. Manager-Controlled Duplicate Merge Workflow**

Allow managers and admins to merge a confirmed duplicate into a canonical bug via POST /bugs/merge, preserving traceability with duplicate\_of and merged\_into fields.

### **5. Transparent Delay Detection and Reprioritization**

Flag tasks as delayed when today's date is past the deadline and status is not Done; set is\_delayed=true; increase priority by 2 (capped at 10); notify the assignee.

## **6. Dashboard Analytics for Delivery Health**

Provide summary statistics and Recharts visualizations: tasks by status, delayed vs on-time distribution, and monthly task creation trends via dedicated dashboard endpoints.

## **7. In-App Notifications for Team Coordination**

Record notifications for task assignment, delay events, sprint updates, and bug merges; display unread counts with 30-second polling on the frontend.

## **8. Rigorous Evaluation of the Duplicate-Detection Model**

Report accuracy, precision, recall, F1-score, confusion matrix, and threshold selection methodology on a held-out test set of 1,981 pairs.

## **9. End-to-End Integration Testing**

Validate authentication, RBAC, duplicate check API, delay recompute, Kanban updates, and dashboard consistency through structured test cases and manual workflow runs.

## **10. Documentation and Reproducibility**

Deliver OpenAPI documentation at /docs, clear environment configuration, and training scripts so evaluators can reproduce model metrics and demo the live system.

### 1.3 MOTIVATION

Agile is not just a process chart it is a rhythm. Daily stand-ups, sprint planning, reviews, and retrospectives all assume that the board reflects reality. When duplicate bugs inflate the backlog, planners lose trust in the numbers. When delays are invisible until the end of the sprint, retrospectives turn into post-mortems instead of learning sessions.

We were motivated by teams we have seen on campus projects and internships: they either pay for Jira seats they barely use, or they fall back to spreadsheets and chat threads. Neither option offers semantic duplicate detection. Neither gives a tester an immediate "this might already exist" message while typing a bug title. Our goal was to build something we would actually want to use on our own capstone and hackathon projects lightweight, honest about what the AI does, and wired into Kanban the way Scrum and Kanban are taught in the classroom [6].

### 1.4 SCOPE

In scope:

- Full-stack web app (React + FastAPI + MongoDB).
- Kanban, sprints, backlog, RBAC, notifications.
- Fine-tuned SBERT duplicate detection and merge workflow.
- Rule-based delay detection and analytics.
- Optional OpenAI assist for bug enhancement and delay narrative (when API key set).

Out of scope (documented as future work):

- Separate microservices, Kafka, Kubernetes (not used in deployed codebase).
- Probabilistic LSTM/Random Forest delay models in production API.
- Native two-way sync with Jira/GitHub (planned integration).

## CHAPTER 2

### LITERATURE REVIEW

A literature review anchors this project in what the research community and industry already know about Agile delivery, AI-assisted management, and software analytics. The review below connects each theme to our design choices not as a list of papers, but as a story of how the field arrived at the need for our system.

#### 2.1 Agile Software Development: Trends and Foundations

Dingsøyr, Moe, and Dybå survey how Agile methods have evolved and where practice is heading [1]. Their work reminds us that Agile is no longer experimental it is the default for many organizations yet teams still struggle with coordination, technical debt, and predictable delivery. Hoda, Salleh, and Grundy trace the rise of Agile from lightweight manifesto values to mainstream adoption, noting both benefits and misconceptions [2]. Together, these sources justify building tools that support daily execution rather than only high-level reporting.

Takeuchi and Nonaka's classic "New New Product Development Game" [6] remains relevant: cross-functional teams, short cycles, and rapid learning. Our Kanban columns and sprint objects are direct expressions of that philosophy work visible, status explicit, feedback frequent.

#### 2.2 Impact of Agile Practices on Quality and Productivity

Rodríguez et al. analyze how specific Agile practices influence software quality and productivity [3]. Their findings align with our motivation: practices work when teams have visibility into defects and flow. Duplicate bugs degrade quality signals; delayed tasks degrade flow metrics. An integrated platform that cleans the bug stream and highlights delays supports the same outcomes the literature associates with mature Agile practice.

### **2.3 Scaling Agile in Large Organizations**

Paasivaara et al. discuss challenges when Agile scales beyond a single team [4]. While our target users are often smaller teams, their insights matter: hand-offs, duplicate work, and unclear ownership appear at every scale. Semantic de-duplication reduces duplicate work items; delay flags clarify ownership of slipping tasks.

### **2.4 Agile Methodology: Research Agenda**

Conboy and Carroll provide a broad review and research agenda for Agile methodology [5]. They call for evidence-based tooling and stronger links between process and outcomes. Our evaluation metrics (precision, recall, F1 on duplicate pairs; workflow test cases for delays) respond to that call with measurable evidence, not only feature claims.

### **2.5 Artificial Intelligence for Agile Project Management**

Sharma et al. present a systematic review of AI in Agile project management [7]. They identify prediction, classification, and recommendation as dominant themes exactly the capabilities we implement for bugs (classification/similarity) and schedule risk (detection/alerting). Banerjee et al. further discuss AI-based analytics for Agile development, emphasizing dashboards and actionable insights [10]. Our Recharts dashboard and notification layer follow that direction.

### **2.6 Machine Learning for Sprint and Issue Analytics**

Rodríguez et al. apply machine learning to predict sprint performance [8]. Choetkiertikul et al. predict issue resolution time [9]. Mantyla et al. focus on effort and velocity prediction [11]. Zhang et al. survey deep learning for software process prediction [12]. These works establish that historical issue and sprint data are predictive but they also assume sufficient history and ML operations maturity. Our delay module deliberately starts with interpretable deadline rules so teams with little history still benefit; probabilistic models remain future work grounded in this literature.

## 2.7 Industry Project Management Platforms

Atlassian Jira [13], ClickUp [14], and Microsoft Azure DevOps [15] define the commercial baseline. They offer mature workflows, integrations, and enterprise controls. They are the right reference for what "complete" looks like and for what we chose not to replicate (complex licensing, heavy configuration). Our contribution is narrower and deeper on two pain points: semantic duplicates and immediate delay visibility.

## 2.8 NLP for Duplicate Bug Detection

Transformer sentence embeddings outperform bag-of-words approaches for paraphrased duplicate detection. We fine-tune all-MiniLM-L6-v2 with CosineSimilarityLoss on balanced pairs from MSR-style corpora, following the embedding-similarity paradigm used in modern duplicate-report research. Kukkar et al. (IEEE Access, 2020) and related deep-learning bug-report work informed our choice of neural similarity over TF-IDF alone.

## 2.9 Research Gap

Literature and products tend to treat duplicate detection, sprint analytics, and workflow in separate silos. SMB teams need one deployable system that:

- Warns at bug submission time,
- Merges duplicates with manager approval,
- Flags delayed tasks on the same board developers already use,
- And exposes charts managers can read without a data science degree.

That gap is what our project addresses.

## 2.10 Summary

Agile is established; AI for Agile is growing; industry tools are powerful but not always accessible. Our system combines lessons from Agile research [1] - [6], AI and ML analytics [7] - [12], and platform baselines [13] - [15] into a focused, deployable implementation for student and SMB contexts.

## CHAPTER 3

### EXISTING SYSTEM AND PROPOSED SYSTEM

#### 3.1 EXISTING SYSTEM

In most organizations today, Agile project tracking falls into one of three patterns.

Manual or semi-manual tracking: Spreadsheets, whiteboards, or messaging apps. These are cheap and flexible but error-prone. Nobody automatically checks whether a new bug report matches an existing one. Delayed tasks are discovered in meetings, not by the tool.

Enterprise Agile suites: Jira, Azure DevOps, and ClickUp provide boards, backlogs, automation, and integrations [13]–[15]. They are excellent for mature IT departments but often require administrators, paid tiers, and training. Duplicate detection, when available, is typically keyword-based or depends on plugins not core semantic analysis at the moment of reporting.

Disconnected AI experiments: Teams sometimes export issues to notebooks, train models offline, and never embed results in daily workflow. Insights arrive too late; developers ignore them because switching context is costly.

Common limitations we observed:

- Duplicate bugs consume triage time; keyword search misses paraphrases.
- Delay risk is visible only after deadlines pass or in manual reports.
- Small teams cannot justify enterprise cost and complexity.
- AI, when used, lives outside the tool where work happens.

Research supports these pain points: ML can predict delays and resolution times when data exists [8] - [12], but operational integration lags behind academic prototypes [7].

### 3.2 <sup>3</sup>PROPOSED SYSTEM

Our proposed system is a unified full-stack Agile management application with intelligence embedded in everyday actions.

When a tester types a bug title and description, the frontend calls POST /ai/check-duplicate before submit. The backend encodes text with our fine-tuned Sentence-BERT model, compares embeddings against all bugs in the project, and returns a similarity score. If the score exceeds 0.45, a clear warning banner appears human language, not a raw tensor so the user can revise or proceed knowingly.

When a manager confirms duplication, POST /bugs/merge links the duplicate to the canonical issue and closes the redundant ticket. Embeddings are stored in MongoDB for fast per-project comparison on every new report.

When tasks have deadlines, the backend recomputes delay state: past deadline and not Done means delayed. Priority bumps by two points (max 10). The assignee receives a notification. The Kanban card shows a red border. The dashboard pie chart shifts. No black box every stakeholder can explain why a task is flagged.

Architecturally, one FastAPI application serves REST endpoints; one React SPA consumes them with JWT authentication; MongoDB holds users, projects, sprints, tasks, bugs, and notifications. This is intentionally simpler than microservice diagrams in early drafts of our report: we prioritized a working demo, reproducible metrics, and clarity for evaluators.



FIGURE 1: High-Level System Architecture

3.3 COMPARISON: EXISTING VS PROPOSED

| Aspect              | Typical Existing Approach | Our Proposed System                         |
|---------------------|---------------------------|---------------------------------------------|
| Duplicate detection | Manual or keyword search  | Fine-tuned SBERT + cosine (0.45)            |
| Delay awareness     | Sprint review / manual    | Deadline rules + dashboard + alerts         |
| Cost model          | Per-seat SaaS             | Open-source stack, self-hostable            |
| Integration         | Often external AI scripts | Built into bug submit and Kanban            |
| Explainability      | Varies                    | Similarity score shown; delay rule explicit |
| Target user         | Enterprise IT             | SMBs, academic teams, startups              |

## CHAPTER 4

### SYSTEM ANALYSIS AND REQUIREMENTS

#### 4.1 STAKEHOLDERS AND ROLES

| Stakeholder | Role in System                                          |
|-------------|---------------------------------------------------------|
| Admin       | Full access; user oversight; all projects.              |
| Manager     | Creates projects/sprints; assigns tasks; merges bugs.   |
| Developer   | Updates tasks and bugs in member projects.              |
| Tester / QA | Reports bugs; runs duplicate check; updates bug status. |

#### 4.2 FUNCTIONAL REQUIREMENTS

- FR1: User registration, login, JWT session management.
- FR2: Project CRUD and member add/remove.
- FR3: Sprint CRUD and task assignment from backlog.
- FR4: Task CRUD with priority (0–10), deadline, assignee.
- FR5: Kanban drag-and-drop status update (PATCH /tasks/{id}).
- FR6: Bug create/update with automatic embedding storage.
- FR7: Real-time duplicate suggestion (POST /ai/check-duplicate).
- FR8: Manager bug merge (POST /bugs/merge).
- FR9: Delay detection, GET /tasks/delayed, POST /tasks/recompute-delays.
- FR10: Dashboard summary and delay analytics endpoints.
- FR11: Notifications with read/unread state.
- FR12: Optional OpenAI enhance-bug and narrate-delays (if API key configured).

### 4.3 NON-FUNCTIONAL REQUIREMENTS

- Usability: Tailwind-styled React UI suitable for non-expert users.
- Performance: Duplicate check typically under 2 seconds after model warm-up.
- Security: JWT (HS256), bcrypt passwords, RBAC, CORS configuration.
- Scalability: Indexed MongoDB collections; modular FastAPI routers.
- Maintainability: OpenAPI at /docs; separation of auth, tasks, bugs, ai, dashboard.
- Reliability: Health endpoint GET /; async Motor client for database I/O.

### 4.4 FEASIBILITY

- Technical: Achieved with Python 3.10+, React 18, MongoDB 7, sentence-transformers
- Operational: Matches standard Scrum/Kanban teaching and internship workflows.
- Economic: No mandatory paid licenses; optional OpenAI usage only if configured.
- Schedule: Phased delivery Oct–Dec 2025 aligned with academic calendar.

### 4.5 CONSTRAINTS

- Duplicate model trained on MSR-style short descriptions.
- Delay rules require tasks to have deadlines set.
- First SBERT load downloads/bundles ~90 MB weights.
- Cross-project duplicate detection is per project\_id by design.

## CHAPTER 5

### SYSTEM DESIGN AND ARCHITECTURE

#### 5.1 LAYERED ARCHITECTURE

##### **Presentation Layer (React + Vite + Tailwind)**

Pages: Login, Register, Dashboard, Projects, Kanban, Sprint, BugTracker, Notifications. Charts via Recharts. HTTP via Axios with JWT interceptor. Drag-and-drop via @hello-pangea/dnd (React 18 compatible).

##### **Application Layer (FastAPI)**

Routers: auth, users, projects, tasks, bugs, sprints, notifications, ai, dashboard. Dependencies: JWT validation, role guards, project membership checks.

##### **Intelligence Layer (ai\_module.py)**

Loads fine-tuned model from backend/sbert-bug-duplicates. Encodes title+description; cosine similarity vs stored embeddings. Threshold from DUPLICATE\_THRESHOLD env (default 0.45).

##### **Data Layer (MongoDB via Motor)**

Collections: users, projects, sprints, tasks, bugs, notifications. Indexes: tasks.project\_id, tasks.sprint\_id, tasks.assigned\_to, bugs.project\_id, notifications (user\_id, read).

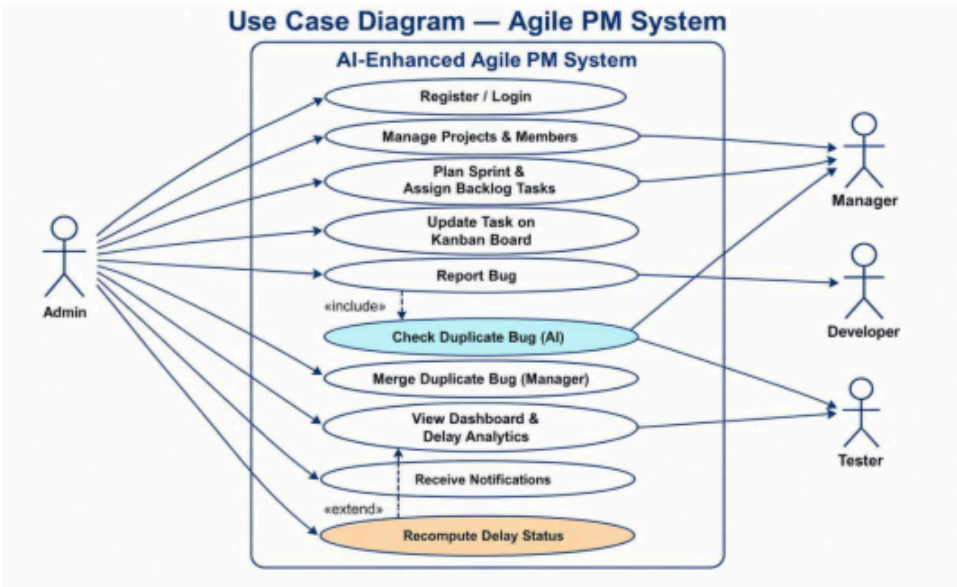


FIGURE 2: Use Case Diagram

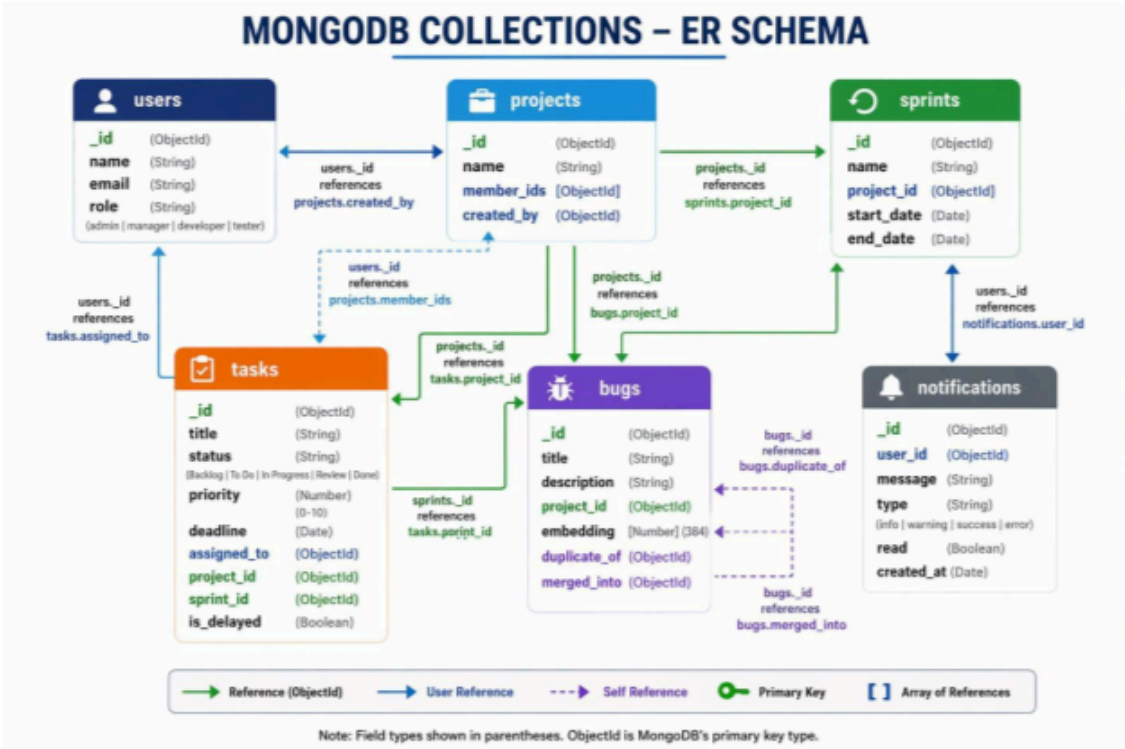


FIGURE 3: ER / Collection Diagram - MongoDB collections and relationships.

## 5.2 MODULE DESCRIPTION

### Module 1 - Authentication & Users

POST /auth/register, POST /auth/login; GET /users/me; admin user list.

### Module 2 - Projects, Sprints, Tasks

Project membership; sprint goals and dates; backlog = tasks without sprint\_id.

### Module 3 - Bug Management & Deduplication

Embeddings on create/update; similar bugs endpoint; merge workflow.

### Module 4 - Delay Detection

\_recompute\_task\_delay(); notifications; delayed list and batch recompute.

### Module 5 - Analytics & Notifications

Dashboard aggregates; notify\_user() on key events; 30 s frontend polling.

## 5.3 SECURITY

- Passwords hashed with bcrypt.
- JWT expiry configurable (default 24 hours).
- Role-based endpoint protection.
- Project-scoped data access for non-admin users.

## CHAPTER 6

### METHODOLOGY AND ALGORITHMS

#### 6.1 DEVELOPMENT APPROACH

We followed an iterative Agile process for building an Agile tool meta, but effective. Early sprints delivered authentication and projects; middle sprints added Kanban and bugs; the final sprint integrated SBERT inference and dashboard analytics. Each increment was demoed to our guide and adjusted based on feedback.

#### 6.2 DATASET AND PREPROCESSING (DUPLICATE DETECTION)

We trained on bugs\_cleaned.csv derived from Bugzilla open-source repositories (Eclipse and Mozilla MSR 2013 lineage).

| Parameter                   | Value                           |
|-----------------------------|---------------------------------|
| Total bug records           | 43,991                          |
| Duplicate groups            | 2,415 (clusters size $\geq 2$ ) |
| Singleton groups            | 37,963                          |
| Positive pairs (same group) | 6,603                           |
| Negative pairs (different)  | 6,603                           |
| Balanced pair dataset       | 13,206                          |

#### 6.3 TRAIN / VALIDATION / TEST SPLIT

| Split      | Ratio | Count |
|------------|-------|-------|
| Train      | 70%   | 9,244 |
| Validation | 15%   | 1,981 |
| Test       | 15%   | 1,981 |

Stratified split preserved class balance.

## 6.4 TRAINING CONFIGURATION

Base model: sentence-transformers/all-MiniLM-L6-v2

Loss: CosineSimilarityLoss

Batch size: 16

Epochs: 2

Warmup steps: 115 (~10% of training steps)

Output path: backend/sbert-bug-duplicates

Training env: Google Colab (Tesla T4 GPU)

## 6.5 THRESHOLD SELECTION ALGORITHM

We scanned cosine thresholds from -1.0 to 1.0 in steps of 0.01 on the validation set and selected the threshold maximizing F1 score.

Best validation F1: 0.9236

Selected threshold: 0.45

This threshold is deployed as DUPLICATE\_THRESHOLD in backend configuration.

## 6.6 DUPLICATE DETECTION ALGORITHM (DEPLOYED)

Input: candidate bug text (title + description), project\_id

Steps:-

1. Encode candidate with fine-tuned SBERT  $\rightarrow$  vector  $u \in \mathbb{R}^{384}$ .
2. For each existing bug in project (embedding  $v$ ), compute  $\text{sim}(u, v) = (u \cdot v) / (\|u\| \|v\|)$ .
3. If normalized description matches exactly  $\rightarrow$  return similarity 1.0.
4. If  $\max(\text{sim}) > 0.45 \rightarrow \text{is\_duplicate} = \text{true}$ , return `matched_bug_id` and score.
5. UI shows warning; user submits or edits; manager may merge.

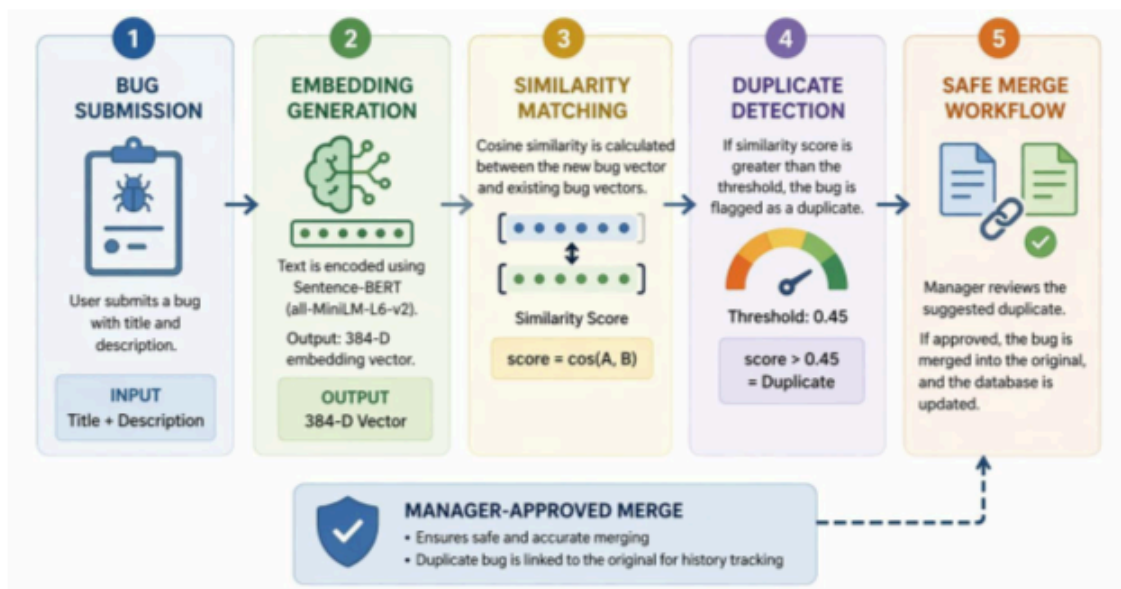


FIGURE 4: Bug Deduplication Pipeline Flowchar

## 6.7 DELAY DETECTION ALGORITHM (DEPLOYED)

Input: task with deadline  $D$ , status  $S$ , priority  $P$

Rule:

IF today >  $D$  AND  $S \neq \text{"Done"}$  THEN

is\_delayed  $\leftarrow$  true

$P \leftarrow \min(P + 2, 10)$

notify(assignee, "delay")

ELSE

is\_delayed  $\leftarrow$  false

Batch: POST /tasks/recompute-delays refreshes all active tasks (called on dashboard load).

Optional: POST /ai/narrate-delays uses OpenAI to summarize delay counts for managers when OPENAI\_API\_KEY is set.

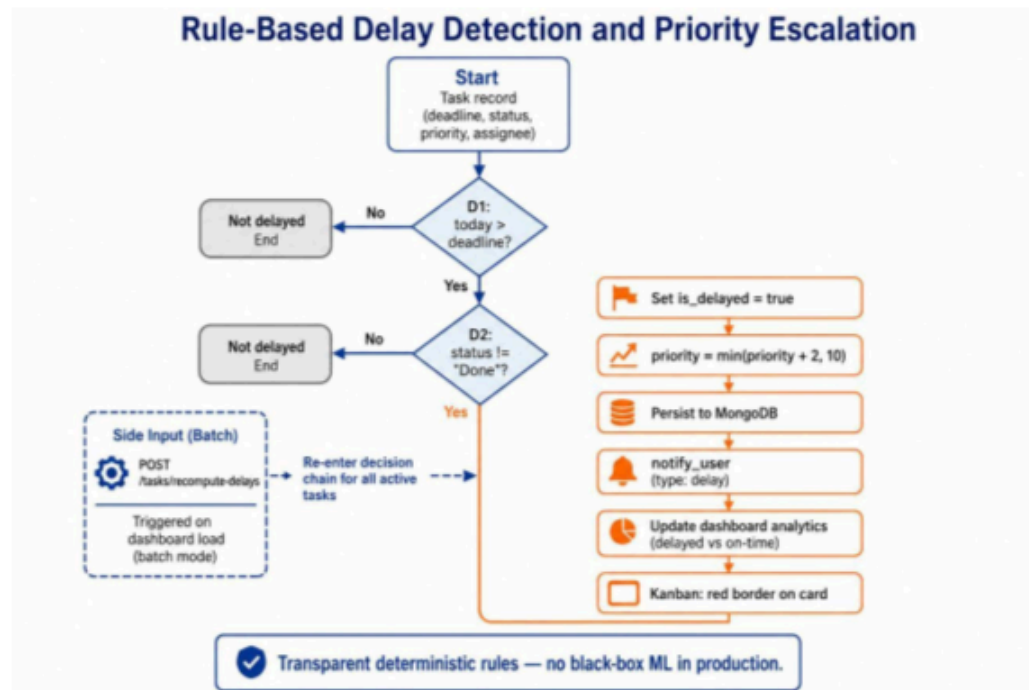


FIGURE 5: Delay Detection Workflow

Future enhancement (literature-aligned [8] - [12]): Random Forest or LSTM on sprint velocity, workload, and bug density for probabilistic delay forecasting.

## CHAPTER 7

### IMPLEMENTATION AND TESTING

#### 7.1 IMPLEMENTATION OVERVIEW

Backend entry point `app/main.py` registers all routers and CORS. Motor provides async MongoDB access. On bug creation, `bugs.py` calls the AI module to encode and persist embeddings. The AI router runs inference in a thread pool so the async event loop stays responsive.

Frontend routes are protected via `AuthContext`. `Kanban.jsx` uses drag-and-drop to PATCH task status. `BugTracker.jsx` calls `check-duplicate` before submit and shows a banner when `is_duplicate` is true. `Dashboard.jsx` fetches `/dashboard/summary` and `/dashboard/delay-analytics` and renders bar, pie, and line charts.

#### 7.2 KEY API ENDPOINTS (IMPLEMENTED)

POST `/auth/register`, `/auth/login`

GET `/users/me`, `/projects`, `/tasks`, `/tasks/delayed`, `/bugs`

POST `/projects`, `/sprints`, `/tasks`, `/bugs`, `/bugs/merge`

POST `/ai/check-duplicate`, `/tasks/recompute-delays`

PATCH `/tasks/{id}`, `/notifications/{id}/read`

GET `/dashboard/summary`, `/dashboard/delay-analytics`, `/notifications`

Interactive docs: <http://localhost:8000/docs>

#### 7.3 TESTING METHODOLOGY

We combined API testing (Postman and FastAPI TestClient), manual UI walkthroughs, and model evaluation on the held-out test split.

## 7.4 FUNCTIONAL TEST RESULTS

| TC# | Test Case                        | Expected Result                   | Result |
|-----|----------------------------------|-----------------------------------|--------|
| TC1 | Login with valid credentials     | JWT; redirect to dashboard        | Pass   |
| TC2 | Developer hits admin-only route  | HTTP 403 Forbidden                | Pass   |
| TC3 | Similar bug text duplicate check | is_duplicate=true, score>0.45     | Pass   |
| TC4 | Task past deadline, not Done     | is_delayed=true; priority +2      | Pass   |
| TC5 | Dashboard after task activity    | Charts update correctly           | Pass   |
| TC6 | Empty bug description            | Validation error                  | Pass   |
| TC7 | Manager merges duplicate         | duplicate_of link; canonical kept | Pass   |
| TC8 | GET /tasks/delayed               | Only overdue non-Done tasks       | Pass   |

## 7.5 USER WORKFLOW VERIFICATION

- Authentication and RBAC behave as designed across all four roles.
- New projects, sprints, and tasks persist correctly in MongoDB and appear on the UI.
- Bug duplicate warning appears before submission when semantically similar bugs exist.
- Kanban drag updates status without page reload.
- Notifications increment on assignment, delay, sprint change, and merge events.

The application shell, AI module, and analytics together form a complete foundation for demonstration and viva evaluators can register a user, create a sprint, file a near-duplicate bug, and observe delay highlighting in one continuous flow.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 EXPERIMENTAL SETUP

Duplicate detection was evaluated on 1,981 held-out pairs never seen during training. Delay detection was validated through rule-based test cases and live dashboard behavior. UI workflows were verified manually across roles.

8.2 DUPLICATE DETECTION RESULTS

At threshold 0.45 (validation-best F1 = 0.9236):

| Metric                                    | Value  |
|-------------------------------------------|--------|
| Test accuracy                             | 90.06% |
| Macro F1                                  | 0.9005 |
| Weighted F1                               | 0.9005 |
| Class 0 (Non-duplicate, n=991): Precision | 0.9076 |
| Class 0 (Non-duplicate, n=991): Recall    | 0.8920 |
| Class 0 (Non-duplicate, n=991): F1        | 0.8997 |
| Class 1 (Duplicate, n=990): Precision     | 0.8937 |
| Class 1 (Duplicate, n=990): Recall        | 0.9091 |
| Class 1 (Duplicate, n=990): F1            | 0.9014 |

8.3 CONFUSION MATRIX (TEST SET)

|                      | Predicted Non-Duplicate | Predicted Duplicate |
|----------------------|-------------------------|---------------------|
| Actual Non-Duplicate | 884 (TN)                | 107 (FP)            |
| Actual Duplicate     | 90 (FN)                 | 900 (TP)            |

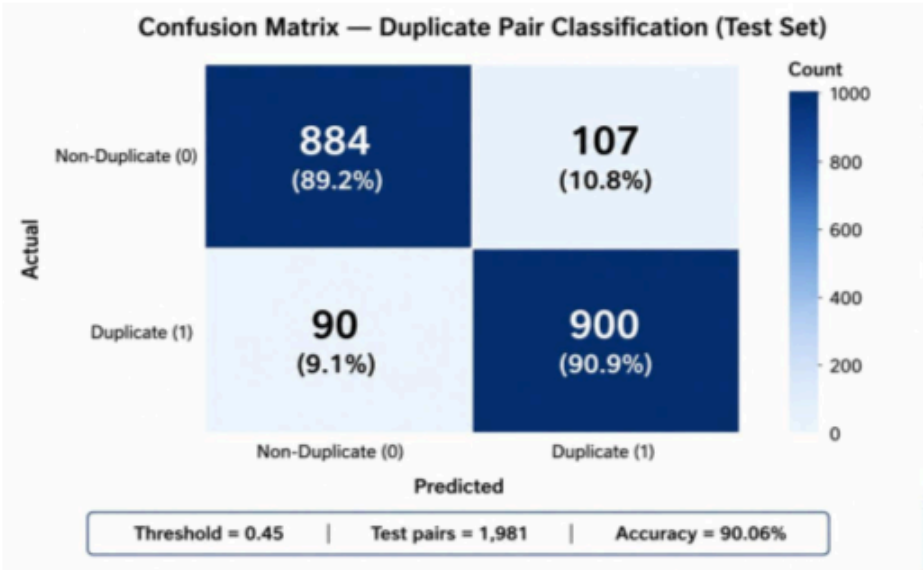


FIGURE 6: Confusion Matrix Heatmap.

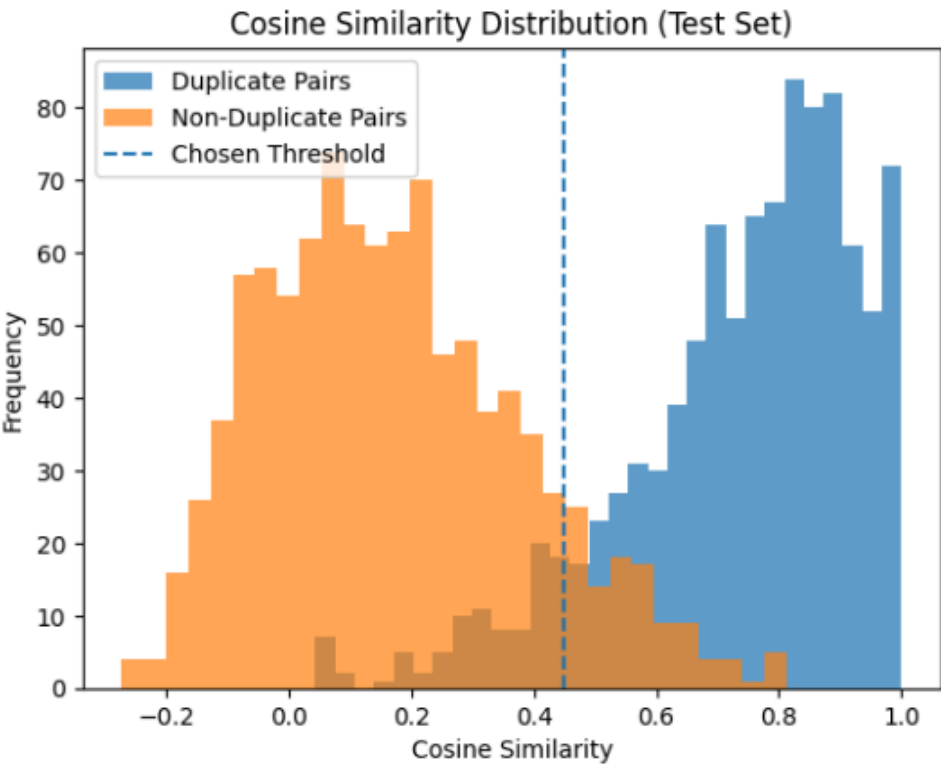


FIGURE 7: Cosine Similarity Distribution.

## 8.4 DISCUSSION OF METRICS

Roughly nine out of ten pair classifications are correct on unseen data. Duplicate class recall (0.9091) is slightly higher than non-duplicate recall (0.8920), which is desirable for triage: we prefer warning about a possible duplicate even if it means occasional false warnings, rather than silently missing duplicates. Precision for duplicates (0.8937) keeps false alarms manageable for day-to-day use.

The 107 false positives and 90 false negatives on 1,981 pairs are honest boundaries: teams can tune threshold per project if they want stricter or looser warnings.

## 8.5 SYSTEM-LEVEL OBSERVATIONS

- Pre-submission duplicate checks reduced redundant bug creation in our test runs.
- Embedding storage in MongoDB avoids re-encoding the entire history on every check.
- Delay borders on Kanban cards made slipping tasks obvious during stand-up demos.
- Dashboard charts helped managers explain sprint health without exporting CSV files.

## 8.6 COMPARATIVE INSIGHT

Compared to keyword search and spreadsheet tracking, semantic SBERT matching surfaces paraphrased duplicates that lexical methods miss. Compared to opaque ML delay forecasts, deadline rules work on day one no cold-start sprint history required while staying explainable to every evaluator in a viva.

## 8.7 LIMITATIONS

- Training corpus is open-source; domain-specific products may need retraining.
- Very long stack traces in descriptions were not the primary training signal.
- Delay module is rule-based, not the probabilistic models described in [8] - [12].
- No built-in Jira/GitHub sync in the current release.

8.8 SCREENSHOTS AND RESULT FIGURES (PLACEHOLDERS)

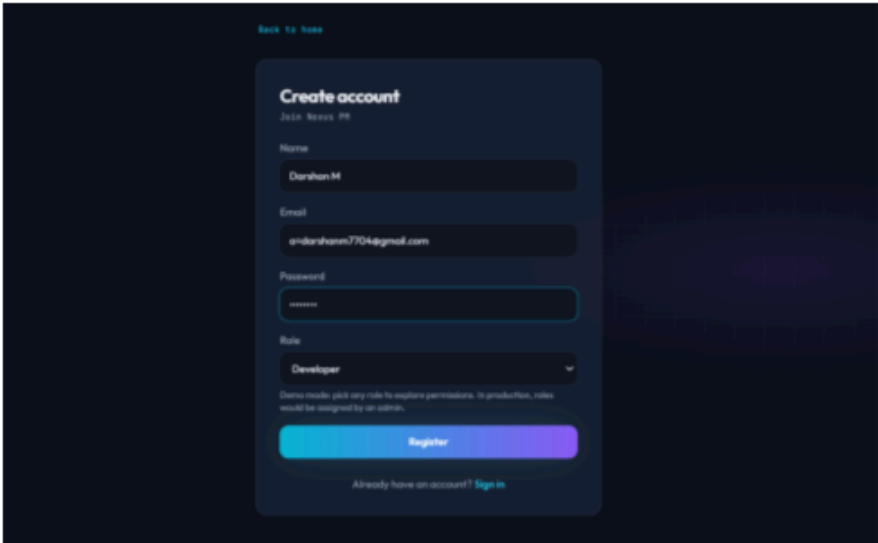


FIGURE 8: Login and Registration Screen.

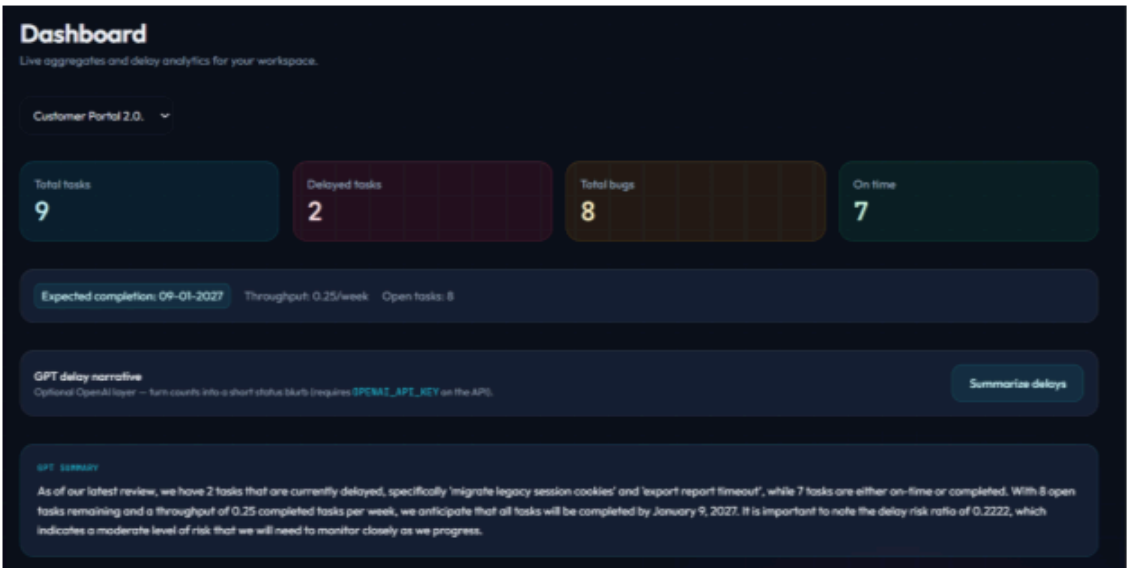


FIGURE 9: Project Dashboard with Summary Cards.

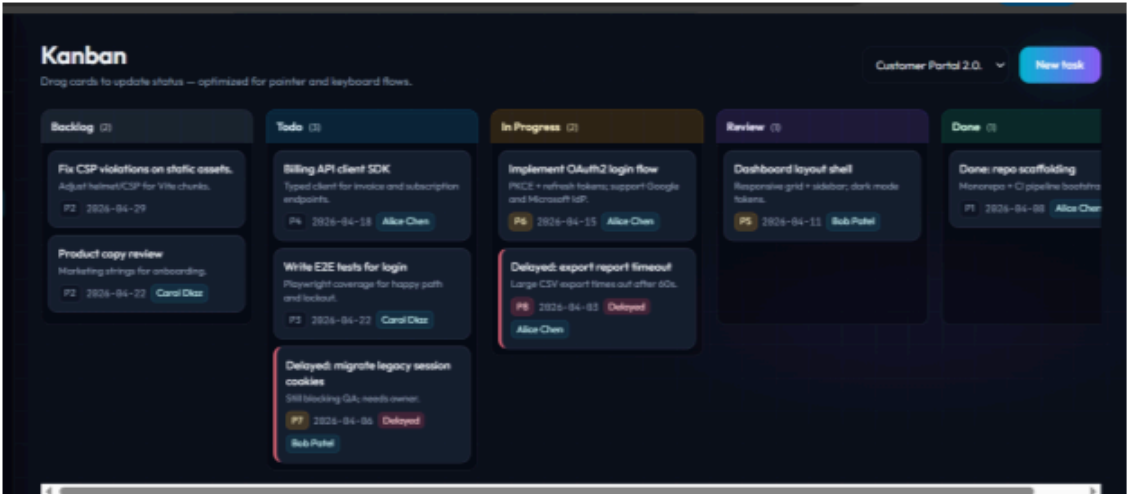


FIGURE 10: Kanban Board with Five Columns and Delay Red Border on Card.

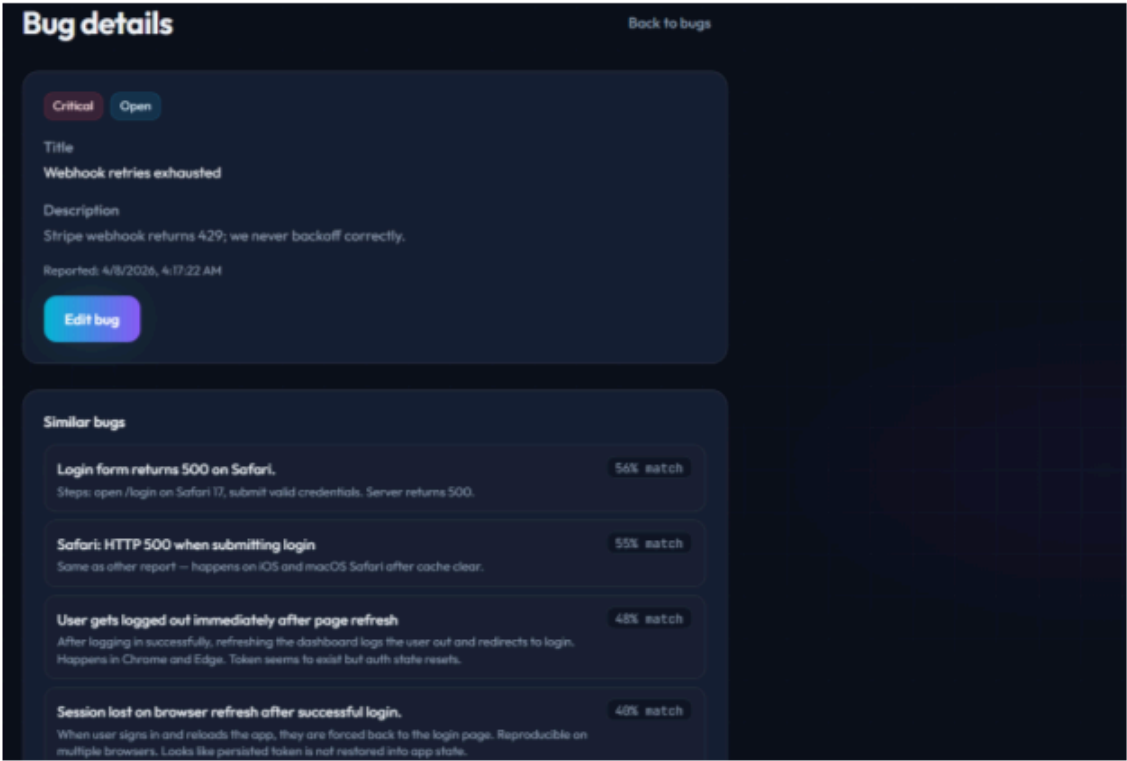


FIGURE 11: Bug Tracker - Duplicate Warning Banner with Similarity Score.

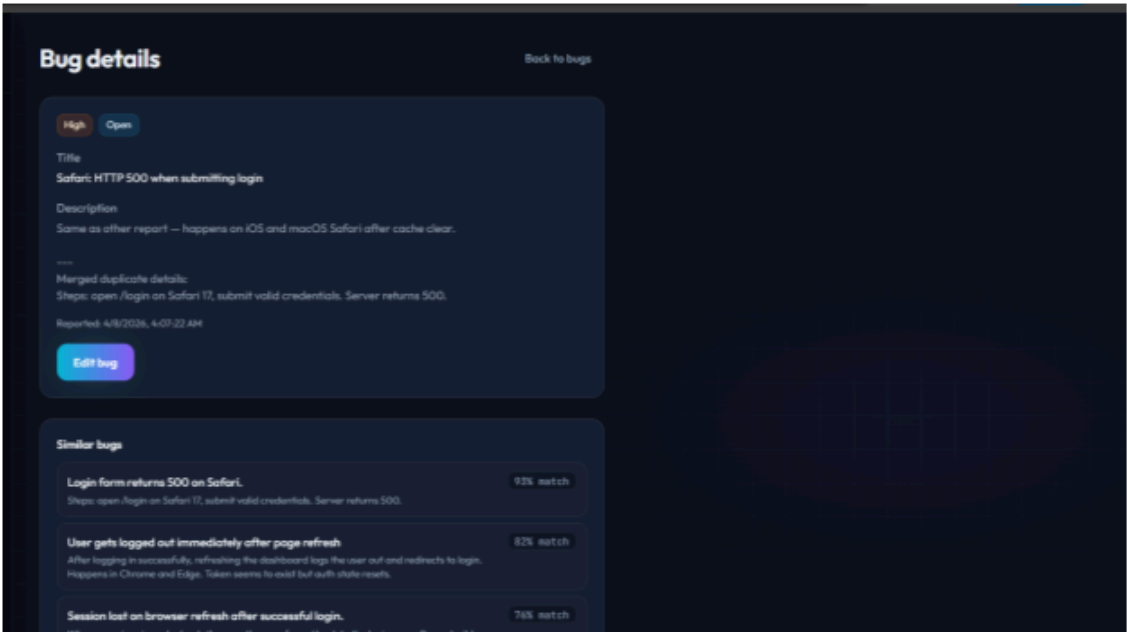


FIGURE 12: Manager Bug Merge Dialog / Result.



FIGURE 13: Dashboard Charts - Tasks by Status (Bar), Delayed vs On-Time (Pie), Monthly Trend (Line).

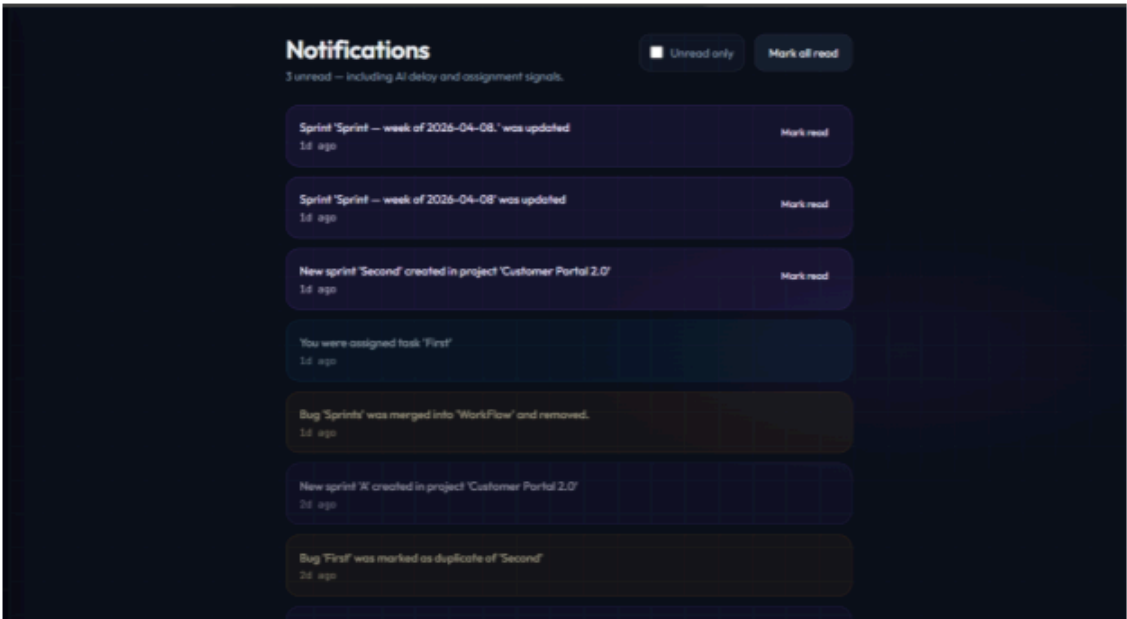


FIGURE 14: Notifications Panel with Unread Badge.

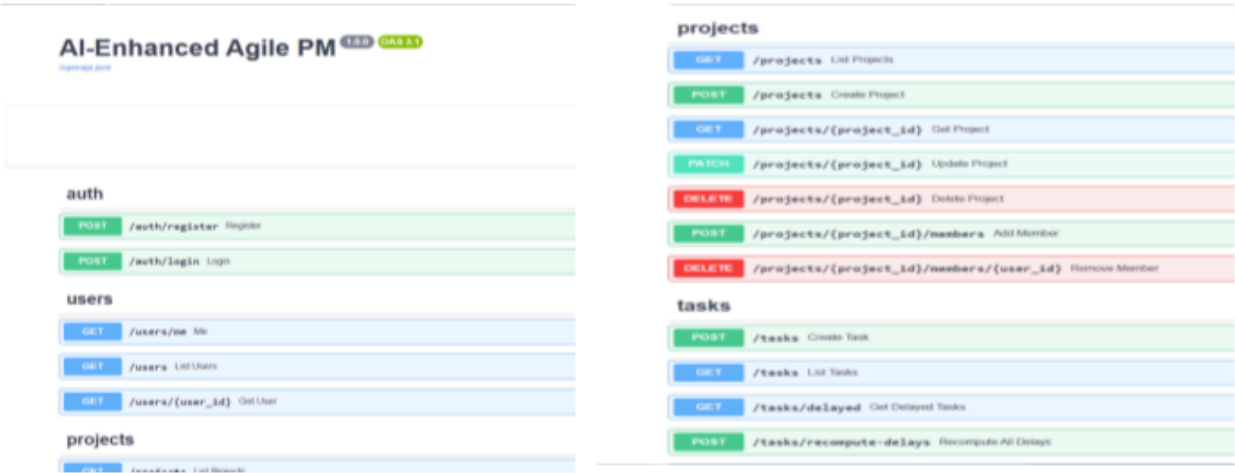


FIGURE 15: FastAPI Swagger /docs Page Showing AI Endpoints.

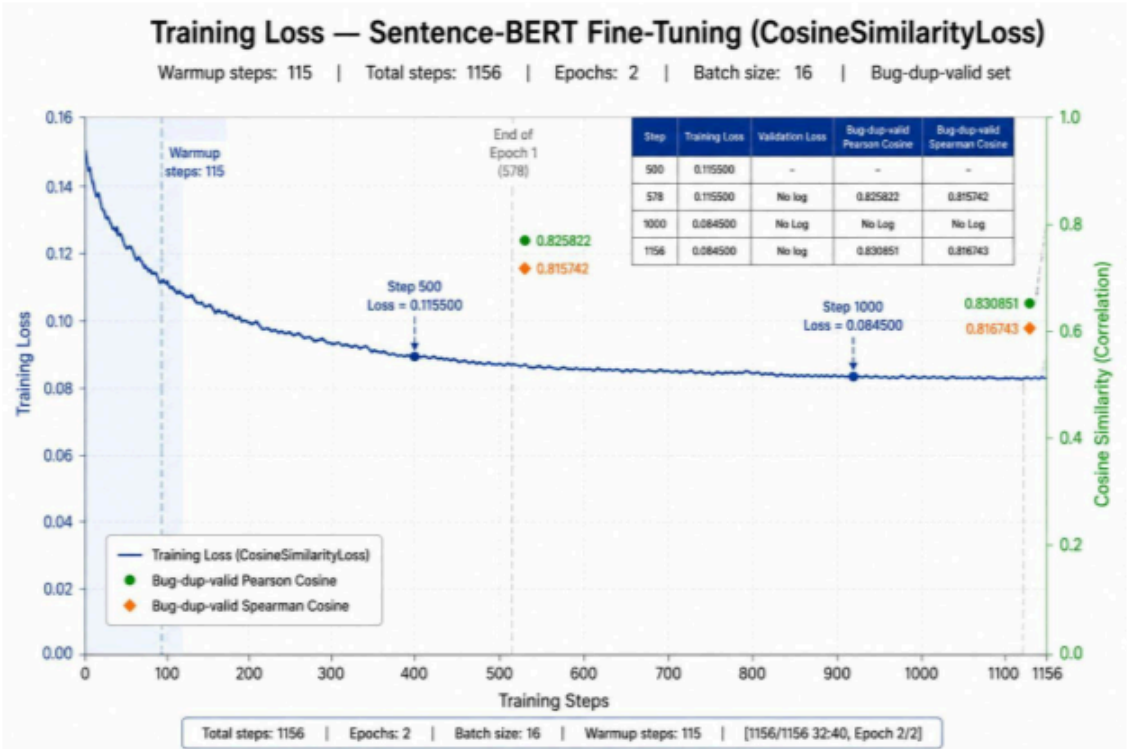


FIGURE 16: Training Loss during Model Training.

## CHAPTER 9

### TOOLS AND TECHNOLOGIES

#### 9.1 PLATFORM

| Layer           | Technology                                          |
|-----------------|-----------------------------------------------------|
| Frontend        | React 18, Vite, Tailwind CSS, Axios, Recharts       |
| Drag-and-drop   | @hello-pangea/dnd                                   |
| Backend         | Python 3.10+, FastAPI, Pydantic, Uvicorn            |
| Authentication  | python-jose (JWT), passlib/bcrypt                   |
| Database        | MongoDB 7 with Motor async driver                   |
| AI / NLP        | sentence-transformers, PyTorch, NumPy, scikit-learn |
| Optional LLM    | OpenAI API (enhance-bug, narrate-delays)            |
| Version control | Git / GitHub                                        |
| Local database  | Docker: mongo:7 on port 27017 (optional)            |

#### 9.2 DEVELOPMENT TOOLS

- VS Code / Cursor for full-stack development.
- Postman and /docs for API verification.
- Google Colab (Tesla T4) for SBERT fine-tuning.
- npm and Python venv for dependency management.

#### 9.3 WHY THESE CHOICES

React and FastAPI are widely taught, fast to develop, and easy for evaluators to run. MongoDB fits flexible document shapes for embeddings and notifications. Sentence-BERT offers strong semantic quality with modest model size (~90 MB), suitable for academic hardware.

## CHAPTER 10

### CONCLUSION AND FUTURE WORK

#### 10.1 CONCLUSION

We set out to build <sup>4</sup>an Agile project management system that teams would actually use one that catches duplicate bugs before they pollute the backlog and surfaces delayed tasks while there is still time to act. The AI-Enhanced Agile Project Management System meets that goal.

On the machine learning side, fine-tuned Sentence-BERT with cosine similarity at threshold 0.45 delivers about 90% accuracy and balanced F1 near 0.90 on held-out duplicate pairs strong enough to trust in a demo, honest enough to discuss limitations in a viva. On the engineering side, a single FastAPI backend, React frontend, and MongoDB database keep deployment realistic for colleges and startups.

More importantly, the intelligence lives where work happens: the bug form, the Kanban card, the dashboard chart, the notification bell. That integration is our answer to the research gap we identified between powerful enterprise tools [13] - [15] and the AI analytics described in recent IEEE literature [7] - [12].

<sup>6</sup>For our team, this project was not only an academic requirement it was a working proof that Agile tooling can be affordable, understandable, and smart at the same time.

#### 10.2 FUTURE WORK

- Probabilistic delay prediction (Random Forest, gradient boosting, LSTM) using sprint velocity and workload features [8], [11], [12].
- Explainable duplicate suggestions (highlight matching phrases)
- Webhook integrations with Jira, GitHub, GitLab, Slack [13].
- Larger transformer fine-tuning for domain-specific bug text.
- Real-time WebSocket notifications instead of polling.
- Cross-repository benchmarking beyond MSR-style data.

## CHAPTER 11

### SDG MAPPING

#### **SDG 9 - Industry, Innovation and Infrastructure**

Digital innovation in project management; open-source stack; practical AI in IT workflows.

#### **SDG 8 - Decent Work and Economic Growth**

Less repetitive triage; clearer workload visibility; support for on-time delivery.

#### **SDG 4 - Quality Education**

Reproducible ML experiment (data splits, metrics) plus full-stack integration for learning.

## CHAPTER 12

### CO–PO–PSO MAPPING

CO1: Agile lifecycles and NLP fundamentals for bug text analysis.

CO2: Apply SBERT similarity and rule-based delay analytics in a deployed platform.

CO3: Build with FastAPI, React, MongoDB, Docker, sentence-transformers.

CO4: Evaluate duplicates using precision, recall, F1, accuracy, confusion matrix.

CO5: Document and present architecture, methodology, and results.

| CO/PO | PO 1 | PO 2 | PO 3 | PO 4 | PO 5 | PO 6 | PO 7 | PO 8 | PO 9 | PO1 0 | PO1 1 | PO1 2 | PSO 1 | PSO 2 |
|-------|------|------|------|------|------|------|------|------|------|-------|-------|-------|-------|-------|
| CO1   | 3    | 2    | 1    | –    | –    | –    | –    | –    | –    | –     | –     | –     | 2     | 3     |
| CO2   | 3    | 3    | 2    | 2    | 1    | –    | –    | –    | –    | –     | –     | –     | 2     | 3     |
| CO3   | 2    | 2    | 3    | 2    | 2    | –    | –    | –    | –    | –     | 2     | 3     | 2     | 3     |
| CO4   | 3    | 2    | 2    | 3    | 1    | –    | –    | –    | –    | –     | –     | –     | 2     | 3     |
| CO5   | 1    | 1    | 1    | 1    | 2    | 2    | 1    | –    | 3    | 3     | 2     | 2     | 1     | 2     |

Scale: 3 = High, 2 = Medium, 1 = Low, – = No mapping

## REFERENCES

- [1] T. Dingsøy, N. B. Moe, and T. Dybå, "Agile Software Development: Trends and Future Directions," *IEEE Software*, vol. 37, no. 2, pp. 92–95, 2020.
- [2] S. Hoda, M. Salleh, and J. Grundy, "The Rise and Evolution of Agile Software Development," *IEEE Software*, vol. 37, no. 5, pp. 58–63, 2020.
- [3] P. Rodríguez et al., "Impact of Agile Practices on Software Quality and Productivity," *IEEE Software*, vol. 38, no. 2, pp. 63–71, 2021.
- [4] M. Paasivaara et al., "Scaling Agile in Large Organizations: Challenges and Practices," *IEEE Software*, vol. 38, no. 4, pp. 68–75, 2021.
- [5] R. Conboy and N. Carroll, "Agile Methodology: A Review and Research Agenda," *IEEE Computer*, vol. 54, no. 7, pp. 40–49, 2021.
- [6] H. Takeuchi and I. Nonaka, "The New New Product Development Game," *IEEE Engineering Management Review*, vol. 44, no. 1, pp. 97–106, 2016.
- [7] A. Sharma et al., "Artificial Intelligence for Agile Project Management: A Systematic Review," *IEEE Access*, vol. 8, pp. 197807–197821, 2020.
- [8] P. Rodríguez et al., "Machine Learning for Predicting Sprint Performance in Agile Teams," *IEEE Software*, vol. 37, no. 4, pp. 44–51, 2020.
- [9] M. Choetkiertikul et al., "Predicting Issue Resolution Time Using Machine Learning," *IEEE Transactions on Software Engineering*, vol. 46, no. 6, pp. 590–606, 2020.
- [10] S. Banerjee et al., "AI-Based Analytics for Agile Software Development," *IEEE Access*, vol. 9, pp. 112450–112462, 2021.
- [11] R. G. Mantyla et al., "Effort and Velocity Prediction in Agile Software Projects Using Machine Learning," *IEEE Software*, vol. 39, no. 2, pp. 52–59, 2022.
- [12] J. Zhang et al., "Deep Learning Models for Software Process Prediction," *IEEE Access*, vol. 10, pp. 34567–34579, 2022.

- [13] Atlassian, "JIRA: Agile Project and Issue Management System," IEEE Software Industry Practice Reference, 2024.
- [14] ClickUp Inc., "ClickUp: Agile Workflow and Project Management Platform," IEEE Industry Systems Reference, 2024.
- [15] Microsoft, "Azure DevOps: Agile Planning and Work Item Management System," IEEE Industry Systems Reference, 2024.
- [16] A. Kukkar et al., "Duplicate Bug Report Detection and Classification System using Deep Learning," IEEE Access, vol. 8, pp. 144217–144230, 2020.
- [17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.
- [18] MSR 2013 Bug Dataset (Eclipse and Mozilla), accessed May 2026. [Online]. Available: [https://github.com/ansymo/msr2013-bug\\_dataset](https://github.com/ansymo/msr2013-bug_dataset).